

Creating Nature Like Patterns using the FitzHugh-Nagumo Model and Fortran

The Maths

The main basis for any two-component reaction-diffusion based experiment, which is what my problem fits into, is two fields which are interacting with each other within some space. Here, there is the variable u which is the *activator variable* and v which is the *inhibitor variable*. The activator component affects the growth of both whilst the inhibitor component slows this same growth (Wikipedia, 2025). My goal was to adjust the parameters of the FitzHugh-Nagumo equation in order to produce Turing Patterns, exciting patterns where stripes and spots can form naturally and remain stable afterward (Wikipedia, 2025). The differential equations can create 6 different states, and I aimed to adjust my parameters in order to create these nature-like patterns - which is why Turing equations are looked at a lot in mathematical biology.

The equations I have used is as follows:

$$\frac{\partial u}{\partial t} D_u \nabla^2 u + f(u, v)$$

$$\frac{\partial v}{\partial t} D_v \nabla^2 v + g(u, v)$$

$$f(u, v) = u - \frac{u^3}{3} - v$$

$$g(u, v) = \epsilon(u + a - bv)$$

In the equations, each component has a diffusion coefficient which has to be precisely chosen in order to create the patterns. To create the patterns I was looking for I needed my inhibitor diffusion coefficient to be very large, in order to create clear areas with little activation. a , b , ϵ and are all reaction parameters and can shape the way the *reaction* occurs between the two components. The laplacian $\nabla^2 u$ represents the spreading of the components, and uses a 5-point stencil to approximate this.

This maths is the basis of my algorithm, and the equation is effectively run for every space in a 300x300 grid over many iterations to simulate a reaction diffusion system.

The Process

I have used Fortran as my language of choice, due to the fact it is compiled, optimised for high-performance scientific applications, and able to handle large numerical arrays efficiently. I also wanted to learn more about the language which has influenced the design of so many other languages. Having been around since 1956, and had an effect on the design of C, Matlab and many more languages used for scientific and engineering applications. In short, the name Fortran which means “Formula Translating System” shows just how suitable it is for the use in this experiment.

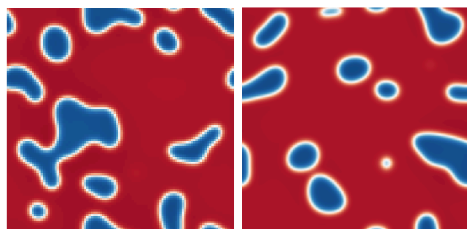
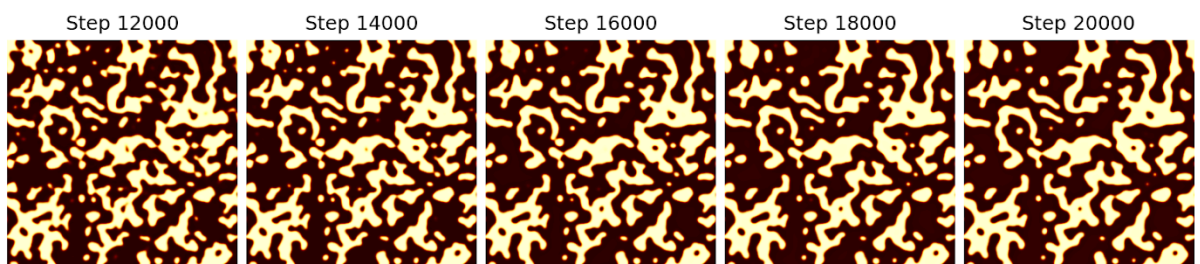
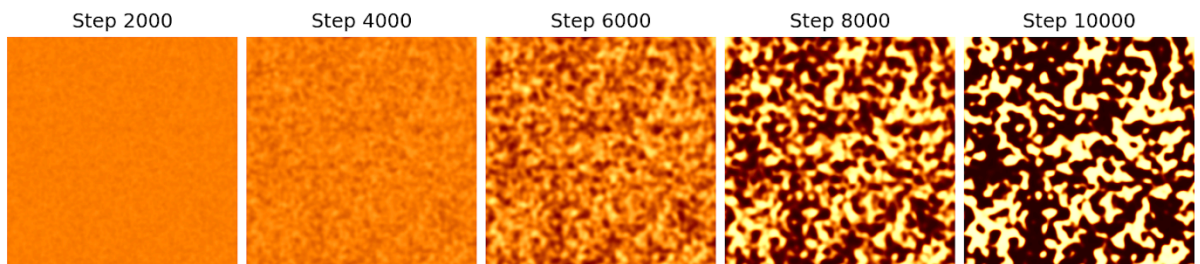
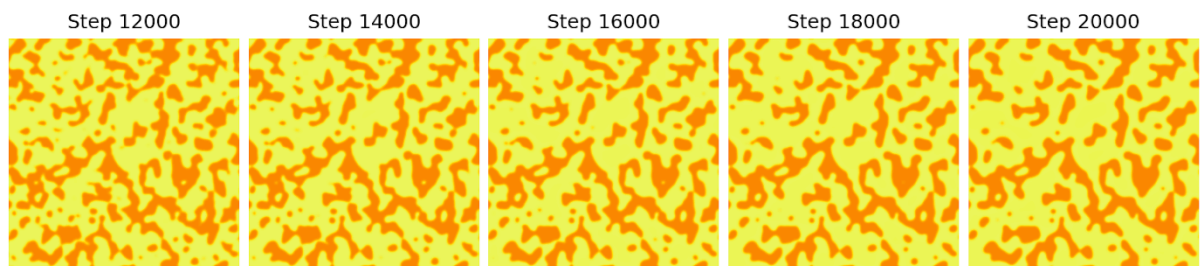
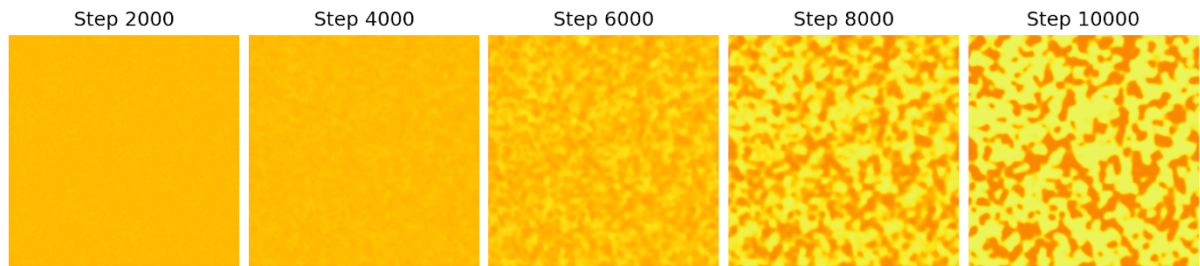
With the language, I can make use of allocating memory early, and letting the heavily optimised compiler do much of the work for me. This makes the algorithm, which requires >10,000 steps run, complete a timeframe which is too short for me to watch even a single TikTok. The first stage is to randomly allocate real number values to indexes around my 300x300 grid, represented using a 2D array. These numbers vary slightly from the equilibrium of the partial differential equation system, and are planted in such a way that my patterns can evolve from a random start.

After this initialisation, I can set the values of my parameters, including a , b , ϵ , $D_u + D_v$ (diffusion coefficients), and Δt . These have been carefully chosen to avoid too much excitation and growth of my membrane potential, which earlier on in my experiment produced a growing wave which was not being suitably inhibited. In order to form these Turing Patterns, I needed my D_v to be much larger than D_u , so that areas of now “growth” could form.

Then the process consists of running a loop of a suitable number of iterations such that the pattern can completely form and stabilise. Within the loop, each space in the grid updates its values according to the partial differential equation, taking in also the values of the 4 points around it for the 5-point-stencil. This continues until the end of the loop, which then creates the pattern inside of the 300x300 grid. Usually the patterns begin to form after 8000 steps, and stabilise fully at around 12000. Then we have a cool image, which I have used Matplotlib in Python to visualise with different kinds of colourmap.

The examples are below!

The Outcomes - Patterns beginning to form and stabilise.



[1] Wikipedia contributors, "Reaction–diffusion system," *Wikipedia, The Free Encyclopedia*, Oct. 6, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Reaction%E2%80%93diffusion_system

[2] Wikipedia contributors, "Reaction–diffusion system," *Wikipedia, The Free Encyclopedia*, Oct. 6, 2025. [Online]. Available: https://en.wikipedia.org/wiki/Turing_pattern